Forelesning 3

mandag 26. januar 2026 08:36

Til nå

- Tensorer
- Prediksjon vs inferens
- Irreducible vs. reducible error
- Regresjon vs. klassifikasjon
- Trening/valid./test sett
- Diskutert over/underfitting
- Bias/varians
- Evalueringsmetrikker
  - klassifikasjon — Confusion matrix, ...
  - regresjon — MSE, MAE, ...

Idag:

- Struktur i et nevralt nettverk — MLP
  - ↳ Input → Hidden → Output
- Nevron, aktivering, vekter, bias
- Ulike lag
- Akt.funksjon, loss-funksjon
- Notasjon,
- At vi kan skrive nettverket som

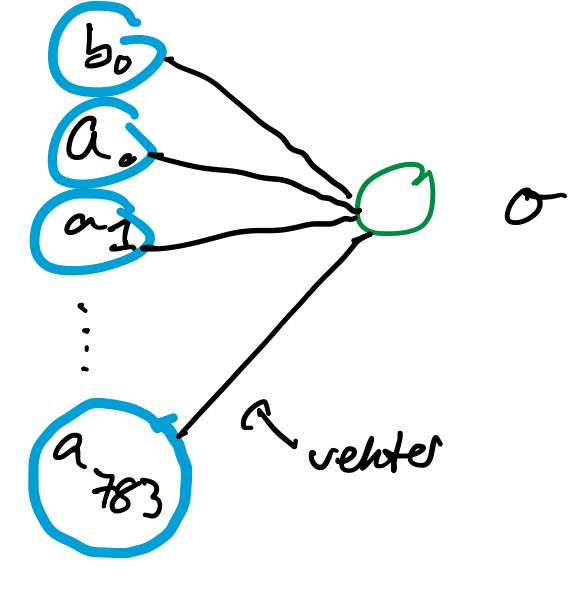
$$f(a_0, a_1, \dots, a_{783}) = \begin{bmatrix} y_0 \\ \vdots \\ y_9 \end{bmatrix}$$

- At "læring" handler om å finne "riktig" verdi for vekter og bias'ene - til å få ønsket prediksjon.
  - ↳ Gode metrikker

Dagens fokus:

Hvordan "lærer" nevrale nettverk?

$\boxed{4}$ → Flates ut til en 784-lang vektor



$\sigma(w_0 a_0 + \dots + w_{783} a_{783} - b_0)$

Slik gjør nettverket det:

- Det gis "tilfeldig" verdi til vektorene ($w_i, i=1,\dots$) og bias (b).
  - ⇒ Da vil (selvsagt) output'et bli dårlig!
  - ↳ Vi trenger en måte å "straffe" funksjonen på. Vi må kunne si "dårlig modell", når den gjør noe galt.

Da defineres vi en kost-funksjon:

Feks. MSE! — tapsfunksjon

Vi skriver ofte kost-funksjon/loss-funksjon som C.

$$Cost = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

Dette evaluerer performance til modellen, og vi har en optimaliseringsalgoritme for å minimere kost.

Liten detalj:

Loss-function: feil for en enkelt prediksjon typ: $(\hat{y}-y)^2$

Cost-function: Gj.snitt/sum av loss funksjoner over hele datasettet.

$$Cost = \frac{1}{n}\sum_{i=1}^{n}(\overset{\text{pred}}{\hat{y}_i} - \overset{\text{observert}}{y_i})^2$$

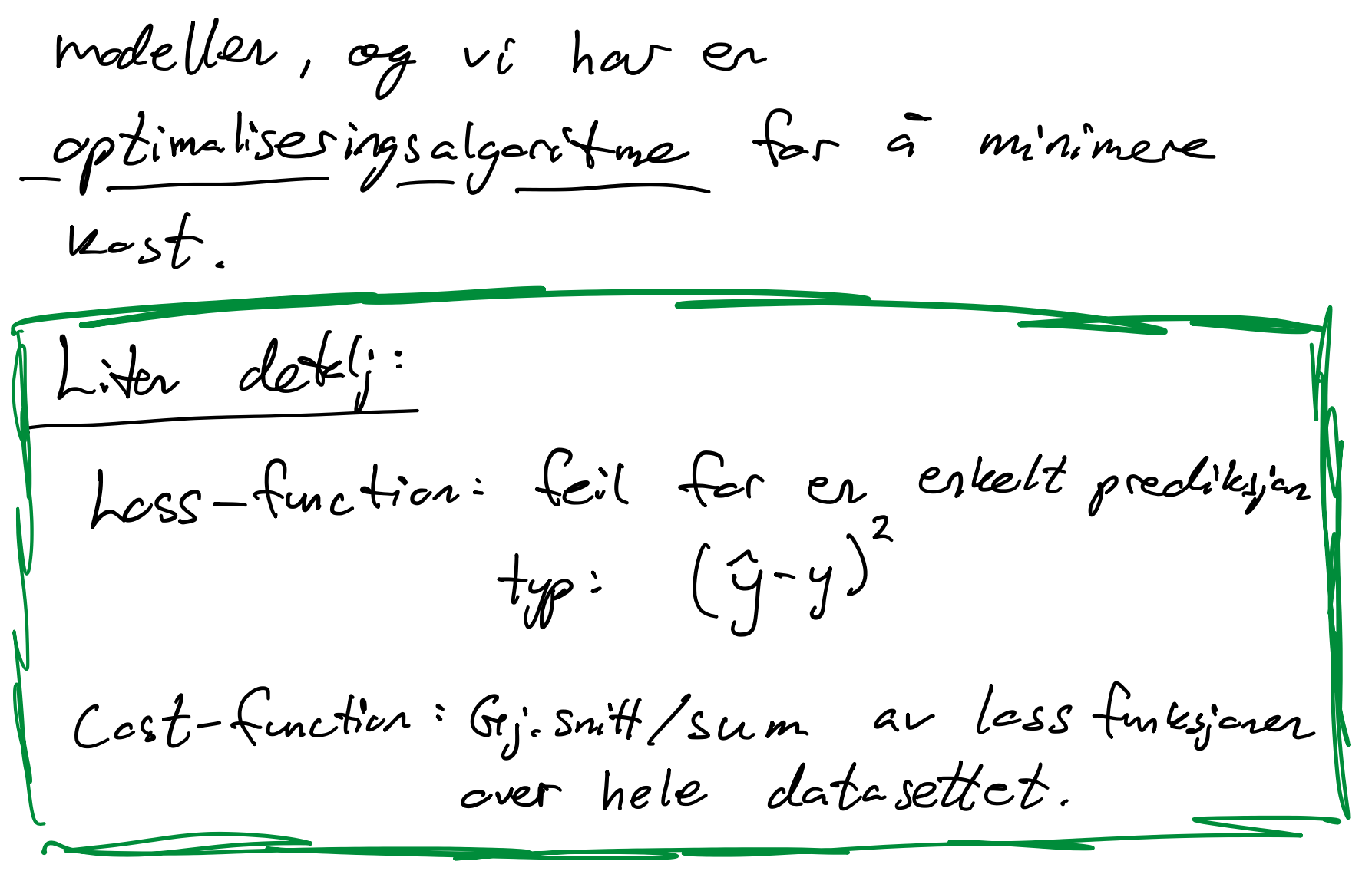
Nettverk-funksjonen	Kost-funksjonen
Input: 784	Input: 13.002 vektor/bias
Output: 10	Output: 1 tall
Parametere: 13.002 vekter og biaser	Parametere: Treningsdata, altså mange eksempler! $(\overset{}{\underset{x_1}{4}},\underset{y_1}{4}), (\underset{x_2}{5},\underset{y_2}{5})$

Vi kan se på cost-funksjonen, og skrive:

$$C(\underbrace{w_1, w_2, \dots, b}_{})$$

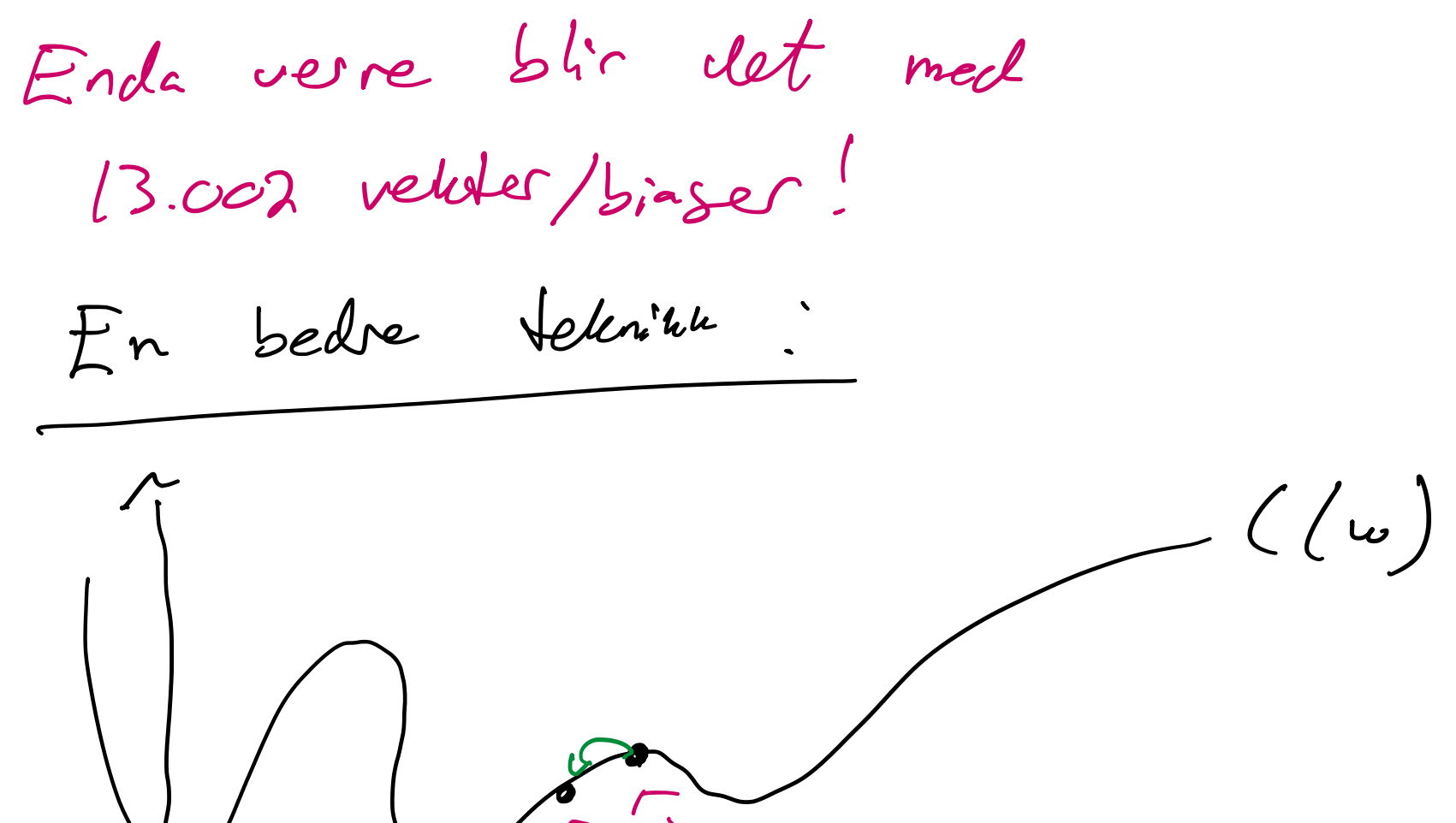
Vår utfordring er å finne de optimale verdiene her!

La oss se på en vekt, ikke 13.002.
Det er lettere å visualisere.



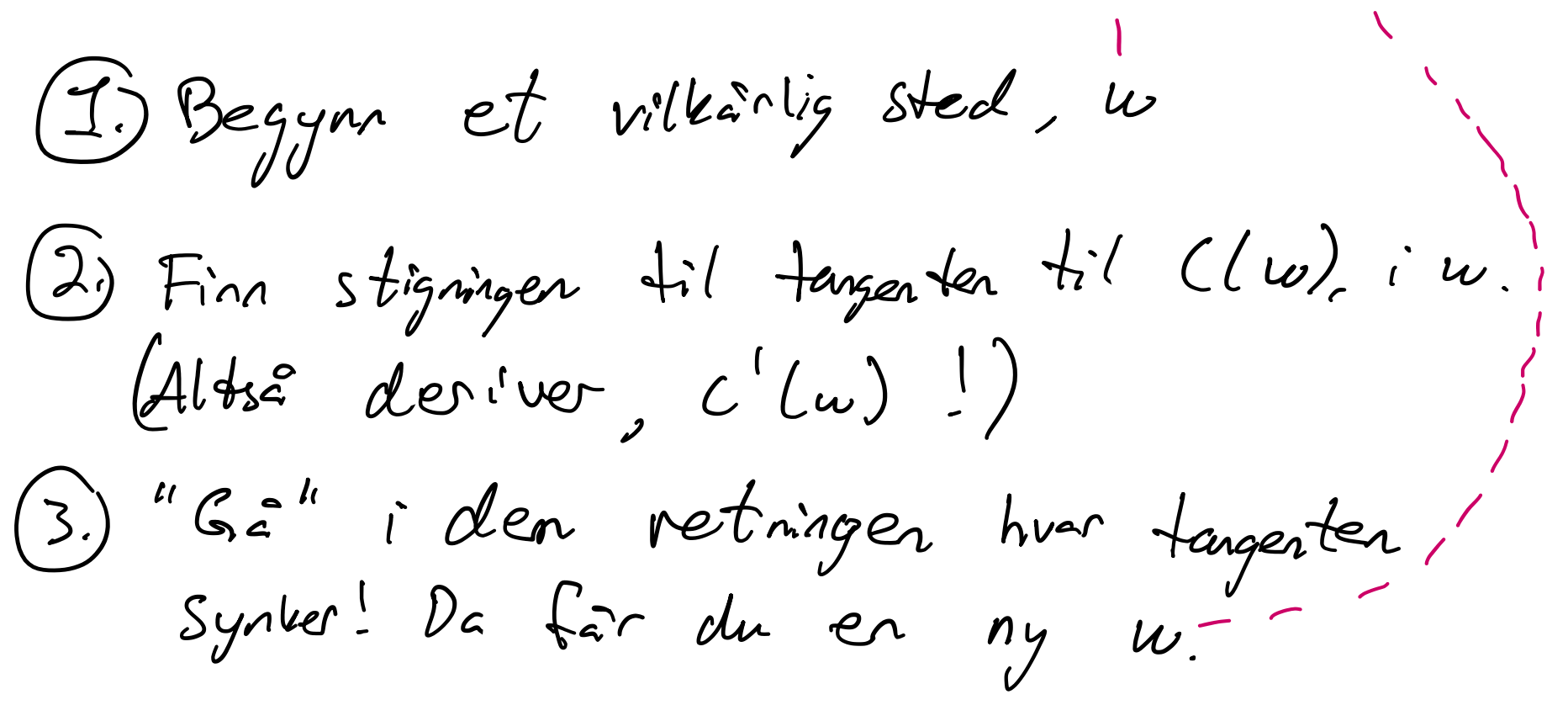
$$\boxed{\frac{dC}{dw}(w) = 0}$$

Dette lar seg ikke alltid gjøre for svært kompliserte funksjoner



Enda verre blir det med 13.002 vekter/biaser!

En bedre teknikk:

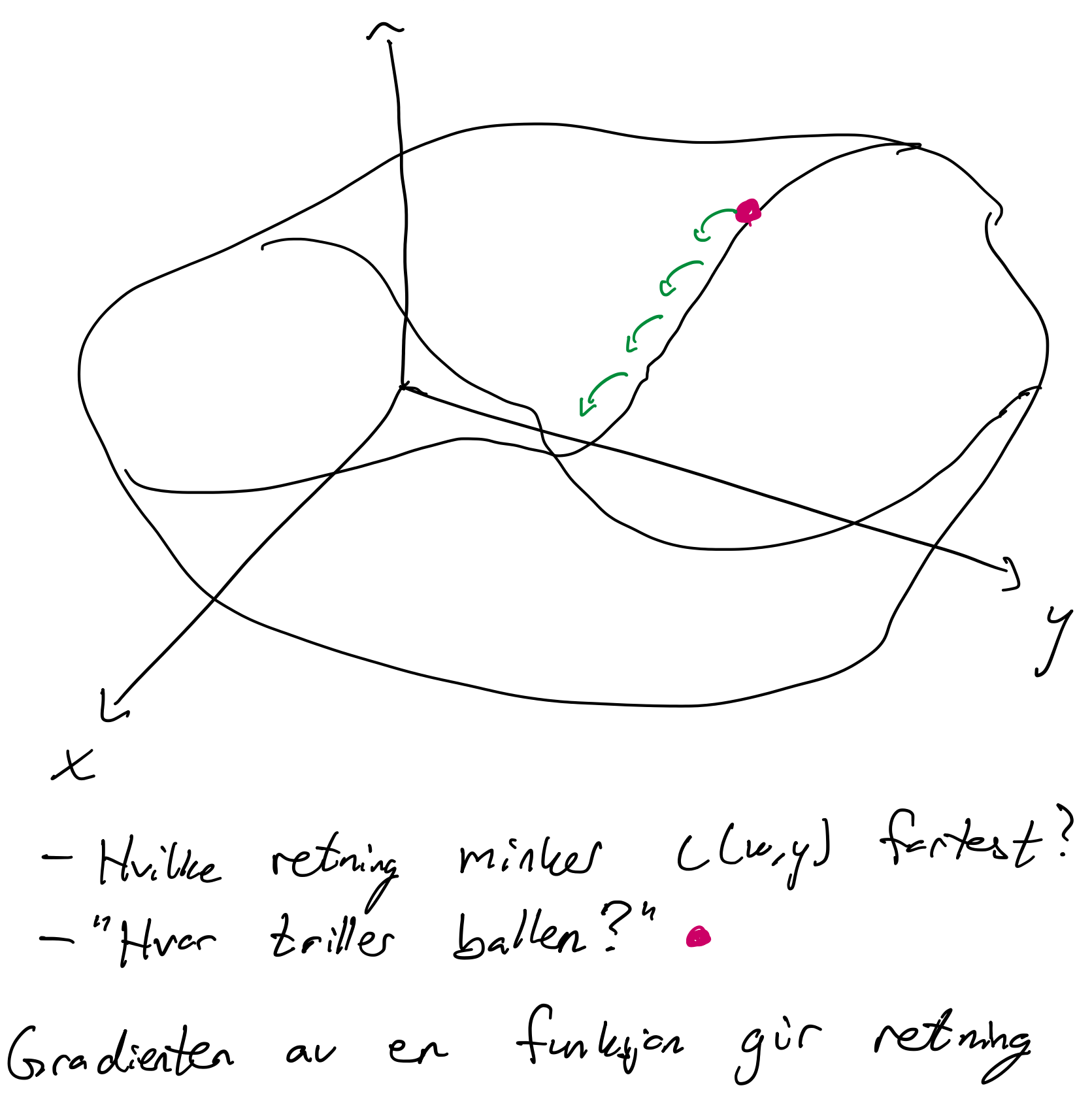


1. Begynn et vilkårlig sted, w
2. Finn stigningen til tangenten til C(w), i w. (Altså deriver, C'(w)!)
3. "Gå" i den retningen hvor tangenten synker! Da får du en ny w.
4. Gjenta 2 og 3, og til slutt havner du i et lokalt minimum.

Obs: lokalt minimum ≠ globalt minimum
Man kan "bli fanget" i lokale minimum.

Kompliserer bildet litt:

C(x,y) som kost-funksjon



- Hvilke retning minker C(x,y) fortest?
- "Hvor triller ballen?"

Gradienten av en funksjon gir retning for brattest stigning. Det skrives slik:

$$\nabla C(x,y)$$

Prosessen vi diskuterer her kalles

Gradient Descent

Gradienten kan regnes ut slik:

$$\nabla C(x,y) = \underbrace{\left(\frac{\partial C}{\partial x}, \frac{\partial C}{\partial y}\right)}_{}, \text{ for to parametere}$$

Denne vektoren gir retningen cost-funksjonen stiger fortest. Vi er da interessert i $-\nabla C(x,y)$!

Dette kan vi gjøre for vilkårlig antall parametere/input!

$$W = \begin{pmatrix} 2.25 \\ 1.62 \\ \vdots \\ -1.41 \\ 2.42 \end{pmatrix} \qquad C(W) = \begin{pmatrix} 0.42 \\ -0.45 \\ \vdots \\ 0.52 \\ -0.41 \end{pmatrix}$$

learningrate - bestemmer "steglengde"!

$$\boxed{W := W - \eta \cdot \nabla C(W)} \text{ ... og repeat!}$$

"Læring" = minimere en kost-funksjon

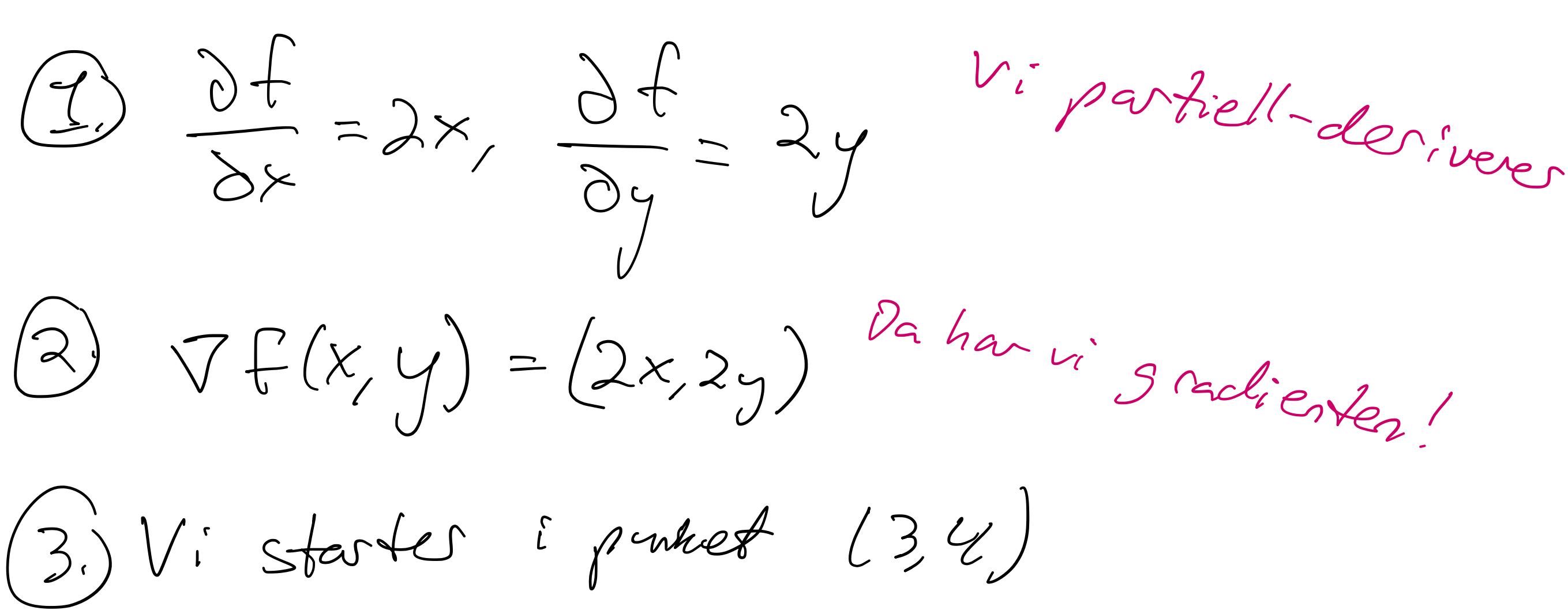
For at dette skal funke, må kost funksjonen være glatt og smooth. Dette er av grunnene til at nettverkets nevroner er kontinuerlig, og i rangen 0 til 1.

Obs: Det er ikke kun minimering av kost-funksjon som skal til for at NN lærer!

For å beregne den "sykt kompliserte" gradienten brukes backpropagation!
↳ En alg. som regner ut $\nabla C(W)$

Ett enkelt eksempel på Gradient Descent:

Vi ønsker å minimere $f(x,y) = x^2 + y^2$



① $\frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 2y$ Vi partiell-deriverer

② $\nabla f(x,y) = (2x, 2y)$ Da har vi gradienten!

③ Vi starter i punktet (3,4)

$(x_0, y_0) = (3,4)$ blått punkt i skisse

Setter læringsrate $\eta = 0.1$

$$(x_1, y_1) = (x_0, y_0) - \eta \cdot \nabla f(x_0, y_0)$$

$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \end{pmatrix} - 0.1 \begin{pmatrix} 2 \cdot 3 \\ 2 \cdot 4 \end{pmatrix}$$

$$= \begin{pmatrix} 3 \\ 4 \end{pmatrix} - 0.1 \begin{pmatrix} 6 \\ 8 \end{pmatrix}$$

$$= \begin{pmatrix} 2.4 \\ 3.2 \end{pmatrix}$$ (grønt punkt i skisse)

Og gjenta for $\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}, \begin{pmatrix} x_3 \\ y_3 \end{pmatrix}, \dots$

Obs: Seminar/kodeøkt vil diskutere

- Regularisering [viktig for prosjekt]
- Sy teori/praksis sammen!